



CLOUDPANDITH ONLINE TRAININGS



Master Data Engineering with Expert-Led Live Training

◇ Azure Data Factory

Complete ETL/ELT

◇ Azure Databricks

Big Data & PySpark

◇ Microsoft Fabric

End-to-End Analytics

📞 ENROLL: **8904424822** | 🌐 www.cloudpandith.com

✓ Live Sessions ✓ Real Projects ✓ Expert Trainers ✓ Interview Prep ✓ Lifetime Access

- ◆ Live Classes
- ◆ Real-Time Projects
- ◆ Interview Preparation
- ◆ Lifetime Access

Index

1. Current challenges with large-scale data processing
2. How Change Data Feed solves these challenges
3. What is Change Data Feed (CDF)
4. When to use CDF and when not to use it
5. Real-time and incremental processing use cases
6. Enabling CDF in Delta Lake
7. Reading changed records using SQL and PySpark
8. Building Audit Trail tables using CDF
9. Building Gold tables incrementally using CDF

Delta Lake Features: Delta Lake Features – Change Data Feed (CDF)

Introduction

Modern data engineering systems process massive amounts of data every day. In real-world enterprise projects, data is continuously changing due to:

- New records being inserted
- Existing records being updated
- Old records being deleted

Processing the entire dataset every time is expensive and inefficient. Delta Lake provides a powerful feature called Change Data Feed (CDF) to solve this problem.

Current Challenges with Data Processing

In traditional data pipelines:

- Data volume grows continuously
- Full table scans become expensive
- Processing takes longer
- Memory and compute costs increase
- Incremental updates become difficult

Example:

Suppose a table contains 500 million records, but only 10,000 records changed today.

Without CDF:

- Entire table must be scanned again

With CDF:

- Only changed records are processed

This significantly improves:

- Performance
- Cost efficiency
- Scalability

How CDF Solves Data Challenges

Change Data Feed helps by:

- Reading only changed data
 - Avoiding full table scans
 - Reducing compute usage
 - Reducing memory consumption
 - Supporting incremental ETL pipelines
 - Sending changes to downstream systems efficiently
-

When to Use CDF

Use CDF	Avoid CDF
Frequent updates and deletes	Append-only datasets
Small percentage of records change	Entire table replaced frequently
CDC data from source systems	Full replace loads
Incremental ETL processing	Full refresh processing
Audit tracking required	Historical tracking not required
Real-time downstream sync	One-time batch loads

What is Change Data Feed (CDF)?

Change Data Feed (CDF) is a Delta Lake feature that captures changes made to a Delta table.

CDF tracks:

- Inserts
- Updates
- Deletes

Instead of reading the entire table repeatedly, we can read only the changed records between versions or timestamps.

Key Features of CDF

Incremental Processing

Read only newly changed records.

Supports CDC Operations

Tracks:

- INSERT
- UPDATE
- DELETE

Better Performance

Avoids expensive full table scans.

Version-Based Processing

Read changes between Delta table versions.

Timestamp-Based Processing

Read changes between specific timestamps.

CDF Metadata Columns

When reading CDF, Delta Lake automatically adds metadata columns.

Column	Description
_change_type	Type of operation
_commit_version	Delta table version
_commit_timestamp	Timestamp of commit

Types of Change Records

Change Type	Meaning
insert	New row inserted
delete	Existing row deleted
update_preimage	Old value before update
update_postimage	New value after update

Real-Time Example

Suppose Employee table contains:

EmpID Name Salary

101 John 5000

Salary updated from 5000 → 7000.

CDF captures:

EmpID Name Salary _change_type

101 John 5000 update_preimage

101 John 7000 update_postimage

This helps us:

- Track history
 - Build audit systems
 - Replicate changes downstream
-

CDF Use Cases

1. Silver and Gold Layer Processing

Instead of rebuilding complete Silver or Gold tables:

- Process only changed records
- Faster ETL execution

- Lower compute cost
-

2. Audit Trail Tables

Maintain complete history of:

- Who changed data
- What changed
- When data changed

Useful for:

- Compliance
 - Governance
 - Debugging
 - Historical tracking
-

3. Materialized Views

Refresh only changed records instead of recomputing entire views.

Benefits:

- Faster refresh
 - Better performance
 - Lower resource usage
-

4. Downstream Synchronization

Transmit only changed data to:

- Data warehouses
 - Reporting systems
 - APIs
 - Analytics platforms
-

5. Real-Time Streaming Pipelines

CDF can integrate with streaming pipelines for:

- Near real-time analytics
 - Event-driven architectures
 - CDC ingestion systems
-

How to Enable Change Data Feed

Method 1 – Enable Globally

```
SET spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;
```

This enables CDF by default for newly created Delta tables.

Method 2 – Enable During Table Creation

```
CREATE TABLE departments (  
  dept_id INT,  
  dept_name STRING,  
  location STRING  
)  
USING DELTA  
LOCATION '/mnt/global/india/landing/departments'  
TBLPROPERTIES (  
  delta.enableChangeDataFeed = true  
)
```

Method 3 – Enable on Existing Table

```
ALTER TABLE departments  
SET TBLPROPERTIES (  
  delta.enableChangeDataFeed = true  
)
```

Reading CDF Using PySpark

```
df = spark.read.format("delta") \  
  .option("readChangeFeed", "true") \  
  .load()
```

```
.option("startingVersion", 2) \  
.option("endingVersion", 6) \  
.table("departments")
```

```
display(df)
```

Understanding the Options

Option	Purpose
readChangeFeed	Enables reading CDF
startingVersion	Starting Delta version
endingVersion	Ending Delta version

Reading CDF Using SQL

```
SELECT *  
FROM table_changes('departments', 2, 5)
```

This reads all changes between:

- Version 2
- Version 5

Example Workflow

Step 1 – Initial Load

ID Name

1 A

2 B

Step 2 - Updates

```
UPDATE departments  
SET dept_name = 'HR'  
WHERE dept_id = 1
```

Step 3 - Read Changes

CDF returns:

- Old value
 - New value
 - Change metadata
-

Building an Audit Trail Table Using CDF

Goal

Store every data modification permanently.

Audit Table Structure

Column	Description
Business Columns	Original table data
change_type	Insert/Update/Delete
commit_version	Delta version
commit_timestamp	Change time

Example Audit Pipeline

```
cdf_df = spark.read.format("delta") \  
    .option("readChangeFeed", "true") \  
    .option("startingVersion", last_version) \  
    .table("departments")
```

```
cdf_df.write.mode("append").saveAsTable("departments_audit")
```

Building Gold Tables Incrementally

Instead of:

Bronze → Full Silver → Full Gold

Use:

Bronze Changes → Silver Changes → Gold Changes

Benefits:

- Faster processing
- Lower cluster cost
- Better scalability

Important Interview Questions

Q1. What is Change Data Feed in Delta Lake?

CDF is a Delta Lake feature used to capture inserts, updates, and deletes incrementally.

Q2. Why is CDF better than full table processing?

CDF processes only changed records, reducing compute and improving performance.

Q3. What metadata columns are added by CDF?

- `_change_type`
 - `_commit_version`
 - `_commit_timestamp`
-

Q4. Difference between CDC and CDF?

CDC	CDF
Source system concept	Delta Lake feature
Captures source DB changes	Captures Delta table changes
Database-level	Lakehouse-level

Q5. When should CDF not be used?

Avoid CDF when:

- Entire dataset gets replaced frequently
 - Data is append-only
 - No incremental processing required
-

Best Practices

Use CDF When

- Data changes incrementally
 - Updates/deletes are frequent
 - Audit history is required
 - Streaming ETL pipelines exist
-

Avoid CDF When

- Entire table reloads frequently
 - Append-only workloads
 - Very high percentage of rows change every batch
-

Advantages of CDF

- Faster incremental processing
- Reduced compute cost

- Better scalability
 - Real-time synchronization
 - Simplified ETL pipelines
 - Easy audit tracking
-

Limitations of CDF

- Requires Delta Lake
 - Additional storage for change logs
 - Not ideal for full replace workloads
 - Version retention depends on Delta history
-

Summary

Change Data Feed (CDF) is one of the most powerful Delta Lake features for incremental processing.

CDF helps:

- Read only changed data
- Build scalable ETL pipelines
- Create audit systems
- Reduce processing cost
- Improve performance

CDF is widely used in:

- Enterprise data engineering
- Real-time analytics
- Medallion architecture
- CDC pipelines
- Streaming systems

Mastering CDF is essential for every modern Data Engineer working with Delta Lake and Databricks.

